

NPRG014 – 2026 plan

5th Oct - Groovy (closures, scripting, LLM integration) (VP)

12th Oct - Groovy (dynamism, DSLs, LLM-backed semantic primitives) (VP)

19th Oct - Groovy (static meta-programming) (VP)

26th Oct – Backup

2nd Nov – Scala (TB)

9th Nov - Scala (TB)

16th Nov - Scala (TB)

23rd Nov – Modern concurrency, AI agentic systems (VP)

30th Nov - Scala (TB)

7th Dec - Prototype-based languages - IO (TB)

14th Dec - Prototype-based languages - JavaScript, TypeScript (TB)

21st Dec - Backup

Criteria

A homework assignment will be given at each lecture

A solution to each homework must be submitted through the Teams project **by the start of the following lecture**

Submit **at least 8 correctly implemented homeworks**

Repository

Clone and then checkout before each lecture

<https://github.com/d3scomp/NPRG014.git>

Communication using Teams

Lesson 1 - prerequisites

- **Laptop**
- **Java**
- **Groovy** - *`groovyConsole`* on cmd runs
- **Ollama** server with a few language models

ollama pull gemma3:4b

ollama pull gemma4

ollama pull gemma4:12b (16 GB RAM min)

ollama pull qwen3.6 (32 GM RAM min)

Closures, scripting and LLM integration



Václav Pech

NPRG014 2026/2027

<http://www.vaclavpech.eu>

@vaclav_pech

SW Developer in the AI era?

- ~~Writing code~~
- ~~Review code~~
- SW Architecture and Design
- Project management
- Product management

NPRG014 – new content

5th Oct - Groovy (closures, scripting, **LLM integration**) (VP)

12th Oct - Groovy (dynamism, DSLs, **LLM-backed semantic primitives**) (VP)

19th Oct - Groovy (static meta-programming) (VP)

26th Oct – Backup

2nd Nov – Scala (TB)

9th Nov - Scala (TB)

16th Nov - Scala (TB)

23rd Nov – Modern concurrency, **AI agentic systems** (VP)

30th Nov - Scala (TB)

7th Dec - Prototype-based languages - IO (TB)

14th Dec - Prototype-based languages - JavaScript, TypeScript (TB)

21st Dec - Backup

Today's agenda

Groovy syntax – **closures**

Scripting

Scripting with LLMs

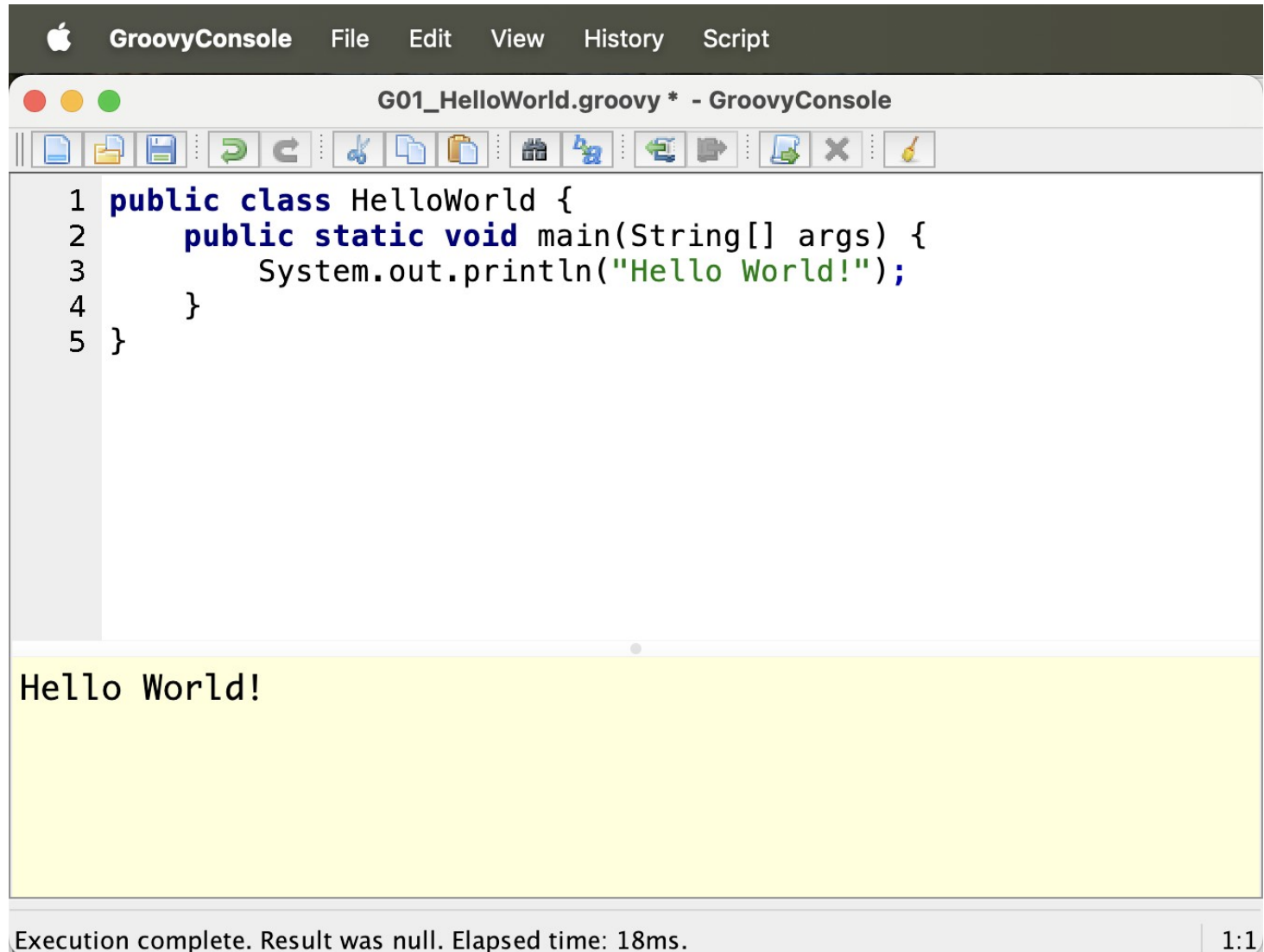
Groovy



A JVM programming language (started 2003)

- Object-oriented
- Building on Java syntax
- Dynamic
- Dynamically-typed
- Scripting (*for Java programmers*)

groovyConsole



The screenshot shows the GroovyConsole application window. The title bar reads "G01_HelloWorld.groovy * - GroovyConsole". The menu bar includes "GroovyConsole", "File", "Edit", "View", "History", and "Script". The toolbar contains icons for file operations (new, open, save, close), editing (undo, redo, cut, copy, paste), and execution (run, stop, clear). The code editor displays a Groovy script with line numbers 1 through 5. The output area shows "Hello World!". The status bar at the bottom indicates "Execution complete. Result was null. Elapsed time: 18ms." and a cursor position of "1:1".

```
1 public class HelloWorld {  
2     public static void main(String[] args) {  
3         System.out.println("Hello World!");  
4     }  
5 }
```

Hello World!

Execution complete. Result was null. Elapsed time: 18ms. 1:1

Why Groovy



Flat learning curve

Concise, readable and expressive syntax, easy to learn for Java developers



Powerful features

Closures, builders, runtime & compile-time meta-programming, functional programming, type inference, and static compilation



Smooth Java integration

Seamlessly and transparently integrates and interoperates with Java and any third-party libraries



Domain-Specific Languages

Flexible & malleable syntax, advanced integration & customization mechanisms, to integrate readable business rules in your applications



Vibrant and rich ecosystem

Web development, reactive applications, concurrency / asynchronous / parallelism library, test frameworks, build tools, code analysis, GUI building



Scripting and testing glue

Great for writing concise and maintainable tests, and for all your build and automation tasks

Why Groovy (in the course)

1. You know **Python**
2. **JavaScript** and **TypeScript** later
3. **Groovy** does things differently

Why Groovy (in the course)

1. You know **Python**
2. **JavaScript** and **TypeScript** later
3. **Groovy** does things differently
4. In the AI era concepts matter, not languages

Long long time ago...

They all use Apache Groovy!



Part 1

Groovy syntax and interoperability

Interoperability

Groovy and *Java* can **implement**, **extend**, **refer** and **call** each other at will.

Groovy sources compile into *.class* files

IDEs provide cross-reference support

Java

```
public class Person {  
    private final String name;  
    public Person(String name) {  
        this.name = name;  
    }  
    public String getName() {  
        return name;  
    }  
}
```

Groovy

```
public class Person {  
    private final String name;  
    public Person(String name) {  
        this.name = name;  
    }  
    public String getName() {  
        return name;  
    }  
}
```

Groovy (optional ;)

```
public class Person {  
    private final String name  
    public Person(String name) {  
        this.name = name  
    }  
    public String getName() {  
        return name  
    }  
}
```

Groovy (optional *return*)

```
public class Person {  
    private final String name  
    public Person(String name) {  
        this.name = name  
    }  
    public String getName() {  
        return name  
    }  
}
```

Groovy (optional *return*)

```
public class Person {  
    private final String name  
    public Person(String name) {  
        this.name = name  
    }  
    public String getName() {  
        name  
    }  
}
```

Groovy (*public* is default)

```
public class Person {  
    private final String name  
    public Person(String name) {  
        this.name = name  
    }  
    public String getName() {  
        name  
    }  
}
```

Groovy (*public* is default)

```
class Person {  
    private final String name  
    Person(String name) {  
        this.name = name  
    }  
    public String getName() {  
        name  
    }  
}
```

Groovy (*properties*)

```
class Person {  
    private final String name  
    Person(String name) {  
        this.name = name  
    }  
    public String getName() {  
        name  
    }  
}
```

Groovy (*properties*)

```
class Person {  
    final String name  
    Person(String name) {  
        this.name = name  
    }  
}
```

Groovy (*named parameters*)

```
class Person {  
    final String name  
    Person(String name) {  
        this.name = name  
    }  
}
```

Groovy > Java

```
class Person {  
    final String name  
}
```

Variables, constants, params

String s - a variable

def s – a variable (type inferred at run-time)

final s – a constant value (type inferred at run-time)

Intuitiveness

Equality $a == b$

Identity $a.is(b)$

() sometimes optional: *println* 'Joe'

String interpolation

```
final s = 'Hi Joe'
```

```
final s = "Hi Dave"
```

```
final s = "Hi $name"
```

```
final s = "Hi ${user.name}"
```

```
final s = """Hi Dave,
```

```
How are you?
```

```
""")
```

Numbers and primitive types

15 - integer

15G - BigInteger

1.5 - BigDecimal

1.5d - Double

All values are objects: 5.upto(10)

Clever boxing and unboxing

Properties

```
class City {  
    String name  
    int size  
    boolean capital = false  
}
```

```
City c1 = new City(name: 'Praha', size: 1200000, capital: true)
```

```
City c2 = new City(name: 'Písek', size: 25000)
```

```
print c1.name
```

```
c2.size = 25001
```

Power assert

assert 5 == customer.score

Exception thrown

17.2.2012 12:30:12 org.codehaus.groovy.runtime.StackTraceUtils sanitize

WARNING: Sanitizing stacktrace:

Assertion failed:

assert 5 == customer.score

```
  | | |
  | | 4
  | [score:4]
false
```

Closures

Closure multiply = {**int** a, **int** b -> **return** a * b}

Closures

```
Closure multiply = {int a, int b -> a * b}
```

Closures

Closure multiply = $\{a, b \rightarrow a * b\}$

Closures – implicit parameter

```
def triple1 = {int number -> number * 3}
```

```
def triple2 = {number -> number * 3}
```

```
def triple3 = {it * 3}
```

Groovy is functional

```
def multiply = {a, b -> a * b}  
def double = multiply.curry(2)  
def triple = multiply.curry(3)
```

```
assert 4 == multiply(2, 2)  
assert 8 == double(4)  
assert 6 == triple(2)
```

Currying vs. Partial application

def multiply = {a, b $\rightarrow$ a * b}

def partial = multiply.curry(3)

def curried1 = {x $\rightarrow$ {y $\rightarrow$ multiply(x, y)}}

def curried2 = {x $\rightarrow$ multiply.curry(x)}

Memoize

```
def func = {a → longComputation(a)}
```

```
def fastFunc = func.memoize()
```

Closure as parameters

```
def foo(String s, Closure c) {...}
```

Closure parameters

```
def foo(String s, Closure c) {...}
```

```
foo('hello', {a, b → a+b})
```

Closure parameters

```
def foo(String s, Closure c) {...}
```

```
foo(,hello', {a, b → a+b})
```

```
foo(,hello') {a, b → a+b}
```

Closure parameters

```
def foo(String s, Closure c) {...}
```

```
foo('hello', {a, b → a+b})
```

```
foo('hello') {a, b → a+b}
```

```
foo('hello') {a, b →
```

```
    return a+b
```

```
}
```

Closure scope

owner

delegate

this

`closure.resolveStrategy =`

`DELEGATE_FIRST / OWNER_FIRST`

`DELEGATE_ONLY / OWNER_ONLY`

Changing closure scope

`closure.delegate = obj`

- changes the closure

`copy = closure.rehydrate(del, owner, this)`

- clones the closure

`with(obj, closure)`

- clones the closure

Collections

```
final emptyList = []
```

```
final list = [1, 2, 3, 4, 5]
```

```
final emptyMap = [:]
```

```
final capitals = [cz : 'Prague', uk : 'London']
```

```
final list = [1, 2, 3, 4, 5] as LinkedList
```

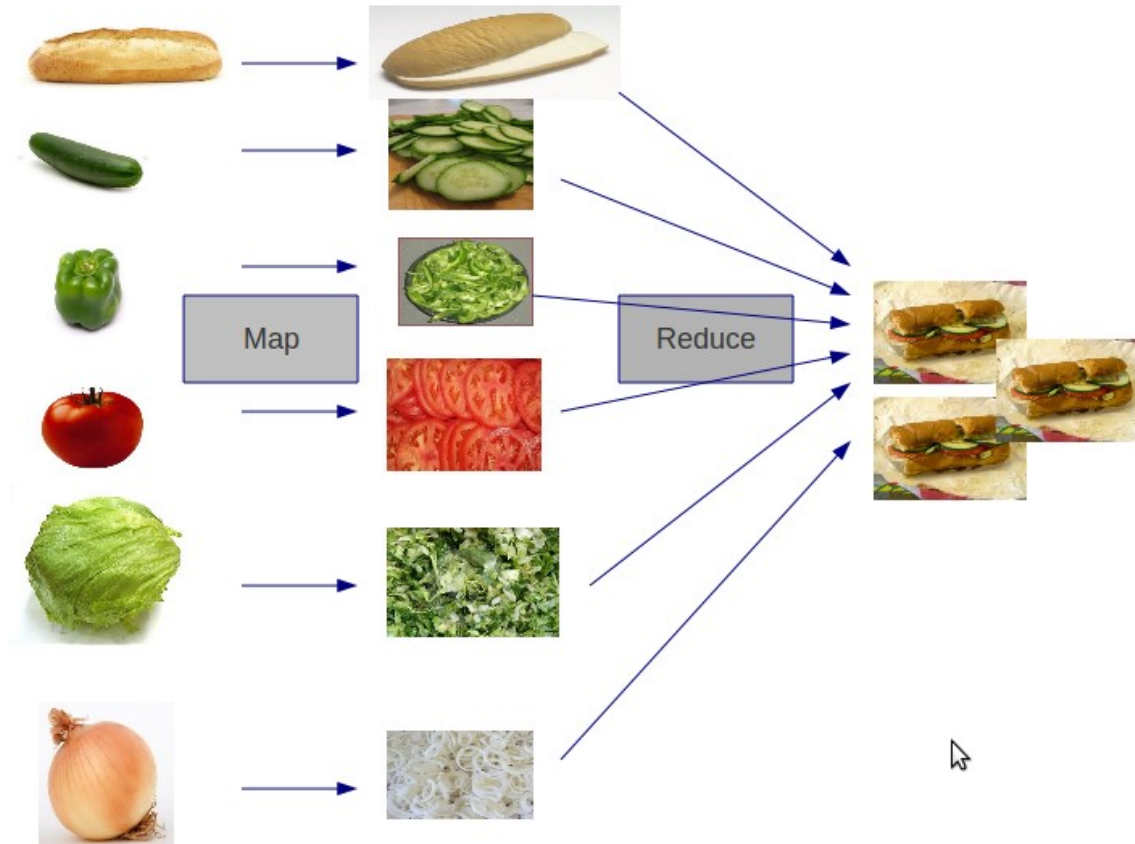
```
final emptyMap =[:] as ConcurrentHashMap
```

Collections API

```
(1..10).each {println it}  
2.step(10, 2) {println it}
```

```
(10..20).findAll{it%2==0}  
    .collect {3*it}  
    .inject(0){acc, v -> acc + v}
```

Filter, map, reduce



Reduce vs. FoldLeft

*String m.reduce {String v1, String v2 →
v1 + v2}*

*int m.foldLeft(0) {int acc, String v →
acc + v.length}*

Reduce vs. FoldLeft

***String m.reduce {String v1, String v2 →
v1 + v2}***

***int m.foldLeft(0) {int acc, String v →
acc + v.length}***

Reduce vs. FoldLeft

Reduce

- *Can run in parallel*
- *Stays within the element type*

FoldLeft

- *Inherently sequential*
- *Not constrained by the element type*

(Not exhaustive) list

each (aka for loop)

collect (aka map)

inject (aka reduce)

findAll (aka filter)

sum, size, findFirst, grep, groupBy

any, every, min, max, ...

Some more operators

```
['Java', 'Groovy']*.toUpperCase()
```

```
customer?.shippingAddress?.street
```

```
return user.locale ?: defaultLocale
```

Part 2

Scripting

Agenda

- Scripting
- Script engine customization

Scripting

Evaluate custom Groovy code

At run-time!!!

```
new GroovyShell().evaluate('println("Hi!")')
```

```
new GroovyShell().parse('println("Hi!")').run()
```

Why scripting?

Runtime configuration

Runtime rule-engine customization

Interpretation of user scripts

Interpretation of LLM-generated code

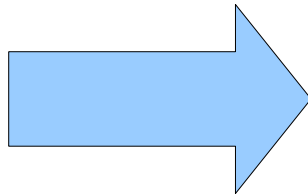
Why scripting?

Runtime configuration

Runtime rule-engine customization

Interpretation of user scripts

Interpretation of LLM-generated code



Security is key

Script customization

CompilerConfiguration

CompilationCustomizer

ImportCustomizer

ASTCustomizer

SecureASTCustomizer

Part 3

Groovy and LLM

G01_LLMGenerateAnswer.gr

```

1 #!/usr/bin/env groovy
2 import helpers.LLMGenerateConnector
3
4 // Prompt for the model
5 def prompt = """
6 What is the capital of CZ?
7 """
8
9 // Use the LLMGenerateConnector
10 def connector = new LLMGenerateConnector()
11 def rawOutput = connector.ask(prompt)
12

```

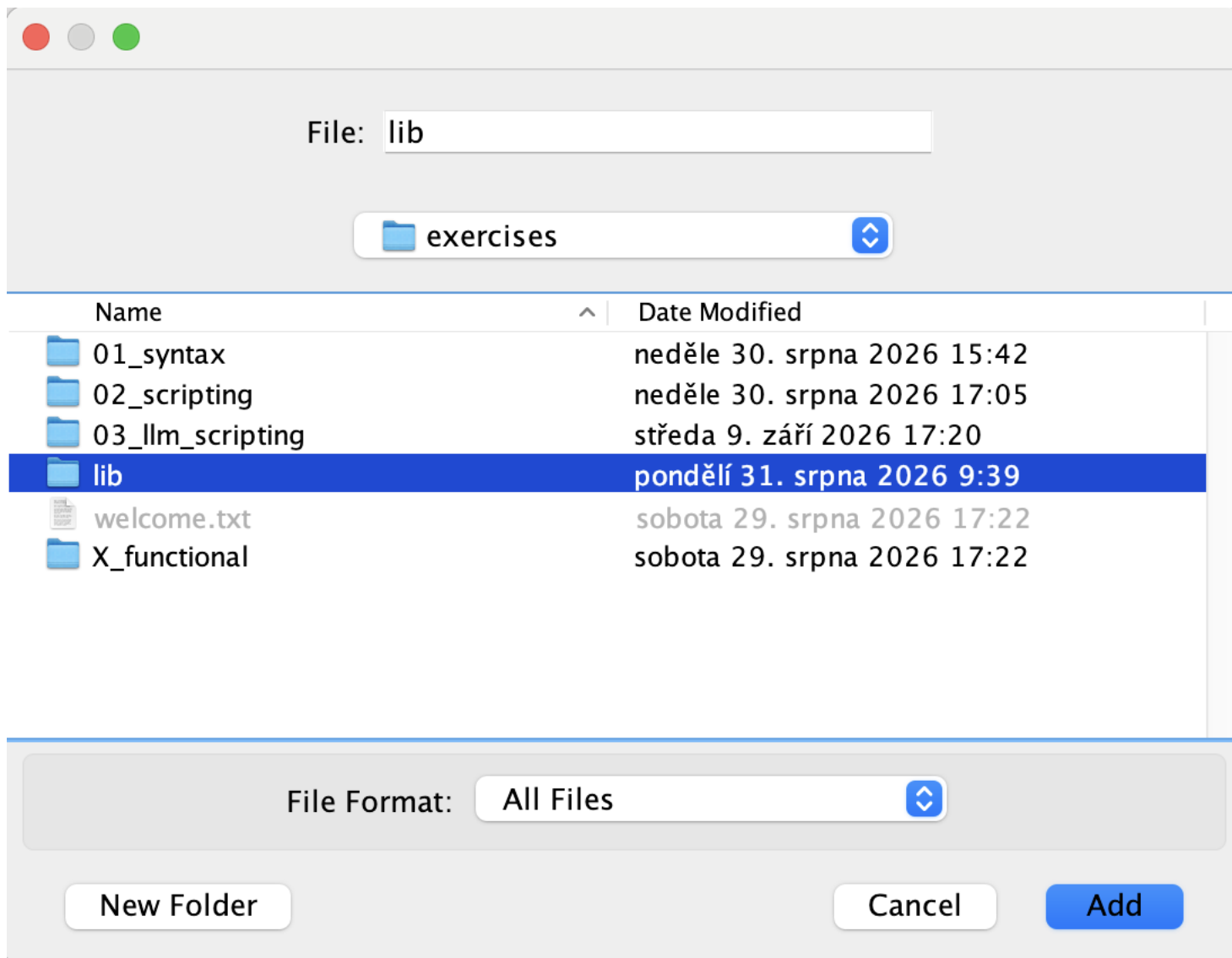
- Run ⌘ R
- Run as Java ⌘ ⌘ R
- Loop Mode
- Auto Save on Runs
- Run Selection ⌘ ⌘ R
- Run Selection as Java
- Allow Interruption
- Interrupt
- Compile ⌘ L
- Compile as Java
- Add Jar(s) to ClassPath
- Add Directory to ClassPath
- List Classpath
- Clear Script Context
- Inspect Last ⌘ I
- Inspect Variables ⌘ J
- Inspect AST ⌘ T
- Inspect CST ⌘ ⌘ T
- Inspect Tokens ⌘ K

1 compilation error:

unable to resolve class helpers.
at line: 2, column: 8

Execution terminated with exception.

1:1



LLM as API

- Text Completions (Generate)
 - Text prompt submitted
 - LLM predicts the chars to follow

LLM as API

- Text Completions (Generate)
 - Text prompt submitted
 - LLM predicts the chars to follow
- Chat Completions
 - Input – messages annotated with roles
 - *system, user, assistant, tool*
 - Output - the next *assistant* message

LLM as API

- Text Completions (Generate)
 - Text prompt submitted
 - LLM predicts the chars to follow
- Chat Completions
 - Input – messages annotated with roles
 - *system, user, assistant, tool*
 - Output - the next *assistant* message

*Both are **Stateless***

Structure of an LLM Chat History

CHAT COMPLETIONS OBJECT (MESSAGE LIST)

 **ROLE: SYSTEM**

"You are a helpful, concise AI assistant specialized in computer science concepts."



 **ROLE: USER**

"Can you explain the context window in simple terms?"



ROLE: ASSISTANT 

"The context window is the total amount of text (tokens) the model can consider at once, including instructions, user queries, and previous conversation history."



 **ROLE: USER**

"Why is it important to manage?"



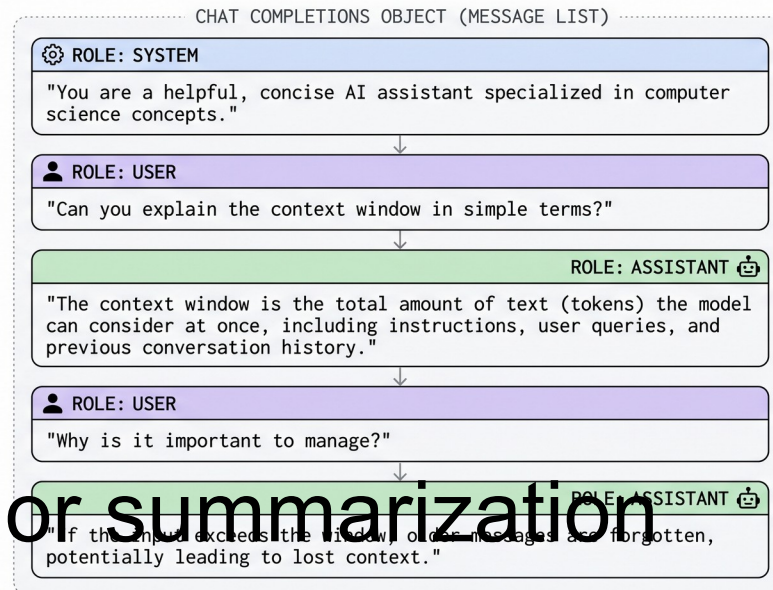
ROLE: ASSISTANT 

"If the input exceeds the window, older messages are forgotten, potentially leading to lost context."

Context Size Reduction

- Sliding window
- Summary
- Selected deletion or summarization

Structure of an LLM Chat History



LLMs specifics for code

- Code snippets typically wrapped

```
``groovy
```

```
numbers.each {println it}
```

```
``
```

Unwrap with a regex

LLMs specifics for code

Use *system* or *user* to set context for the desired code:

- The expected contents of *Binding*
- The desired top-most code element
 - Expression
 - Statement
 - Method definition
 - etc.
- Provide examples of expected output

Summary

Groovy syntax

Scripting

Scripting with LLMs

References

<http://groovy-lang.org>

<https://ollama.com/>

<https://docs.langchain4j.dev/>

<https://www.jetbrains.com/koog/>